



ENHANCING DATA QUERY EFFICIENCY WITH GRAPHQL APIS: A COMPARATIVE STUDY WITH RESTFUL WEB SERVICES

Sudeesh Goriparthi

Senior software engineer, software architecture, Walmart, Dallas, USA.

ABSTRACT

A "stateless architecture design" is a kind of web architecture that, as a general rule, does not use databases to store any user data. Additionally, backup storage is not necessary for apps that match this category. Data in transit is the term used to describe information that is transmitted via a stateless service; nobody ever stores this data. It is not required that processing requests arriving at such an architecture depend on any data collected or saved from any previous session. Applications' data can be accessed using subroutines, definitions, and procedures that makeup what is known as an application programming interface (API). Applications communicate with each other through these points. Clients transmit all the required information to the server in each request, and stateless design makes API endpoint administration easier than traditional architecture. This is because the server side does not retain the client session. Two approaches that can be used in creating this API architecture are RESTful services and GraphQL. Learn the ins and outs of GraphQL and REST API architectural management in this comprehensive essay. Also included is an examination of the potential advantages of GraphQL over REST in planning stateless architectural APIs.

Key words: Application Programming Interface (API), GraphQL, REST

Cite this Article: Sudeesh Goriparthi. Enhancing data query efficiency with GraphQL APIs: A comparative study with RESTful web services. *International Journal of Artificial Intelligence & Machine Learning (IJAIML)*, 3(1), 2024, pp. 132-143.

https://iaeme.com/MasterAdmin/Journal_uploads/IJAIML/VOLUME_3_ISSUE_1/IJAIML_03_01_013.pdf

INTRODUCTION

Since the inception of API-based web services—which were created to enable the seamless integration and exchange of data between applications based on stateless architecture—REST-based API web services have been the architectural style of choice for building and managing web services. Even though there are cases where factors like the speed of a web service API, the specific requirements of the data consumer, and the ease of retrieving data from a single API endpoint influence the choice to use GRAPHQL instead of REST for web service

computing, there is solid evidence to suggest that GraphQL is the better alternative [1]. An alternate architectural style for API-based stateless application architectures could emerge from GraphQL, and this study aims to compare and contrast several possible futures in this area [2]. The goal of our social media post experiment was to test hypotheses with varied amounts of data and then make conclusions. To demonstrate how GraphQL and REST react behaviorally in terms of throughput, we have repeated the experiment multiple times, increasing the data volume with each iteration to account for the complexities of the number of API endpoints.

What is GraphQL?

It is a data query language that you may use to work with APIs, and it is open-source. When it comes to relational databases and Nodes, which are also referred to as objects, GraphQL is optimized. Creating a basic concept that can be utilized for a wide variety of applications in today's data-rich world is the result of the arrangement. Because GraphQL can access numerous data items with a single query, it requires fewer API calls than other query languages. As a result, it is a popular choice for the development of a wide variety of applications.

By providing clients with a single entry point to which they can connect, the application is liberated from the requirements of directly connecting to various services. GraphQL has been embraced by startups such as Airbnb, Shopify, Expedia, and GitHub across a variety of platforms, including the web, mobile, iOS, and Android.

Fundamentals

Objects are represented by nodes, which are defined through the use of the GraphQL schema platform. Edges, on the other hand, are the connections that exist between the nodes in the network. These connections, in turn, define a particular relationship between the queries.

GraphQL gives the client the ability to interact with the server by sending HTTP requests and receiving replies in return. With this data query language, you also have the option of specifying the specific data type that you want to retrieve from the server. It is consequently extremely readable and efficient because the data structure is predictable.

Pros

The reason that GraphQL is faster than any other communication API is that it allows you to reduce the amount of time spent on your request inquiry by allowing you to select only the fields that you wish to query.

By consolidating many microservices into a single GraphQL schema, it facilitates the management of communication between those microservices. This results in a significant reduction in the amount of effort that is necessary to implement any query.

Because GraphQL only retrieves specific data in a single request, you will never experience any issues with over-fetching or under-fetching when using this language.

It is appropriate for the development of a parallel application programming interface (API) for the front end and back end of an application in any programming language.

Reflective in nature is the nature of GraphQL. You can establish a strong platform for developing tools and client applications such as Relay, application frameworks, and integrated development environments (IDEs) like GraphiQL by querying its server for the kinds that it supports.

Upgrades to GraphQL do not have any negative impact on previous infrastructures. Even if the older servers might be deprecated in the future, you are free to continue using them without encountering any difficulties.

Cons

It does not have any built-in capabilities for caching.

Databases that are hosted on the server are not going to be replaced by GraphQL.

When working with GraphQL, it can be difficult to set the maximum amount of requests that can be processed in a single day.

It is challenging to design a simple cache with GraphQL because each query is unique, even though it acts on the same entity.

As one of the three main cloud computing service models nowadays, software as a service (SaaS) has been getting a lot of attention recently. Two examples of such concepts are PaaS and IaaS, or Platform as a Service and Infrastructure as a Service, respectively [3]. Using the internet, software as a service (SaaS) distributes applications to users. A supplier controls these programs. So, it's a big deal to choose the best SaaS provider out of all the ones out there [4]. The technical and functional challenges posed by cloud applications, including multi-tenancy, redundancy, recovery, and scalability, must be met by software architects.

They are responsible for developing software as a service (SaaS) that is both dependable and efficient. Microservices architecture is a distinct trend that has emerged in both academia and industry [6], where significant firms all over the world have advanced their applications towards this architectural style [7]. This trend is intended to help solve the issues that have been being faced.

Small programs with a single responsibility (functional, non-functional, or cross-functional requirements) and the ability to be deployed, scaled, and tested independently make Microservices Architecture (MSA) [8].

Using a variety of programming languages and product stacks, microservices can be independently developed and changed, which enables them to offer agility [9]. The most important benefits that it offers are its ability to be maintained, reused, scaled, made available, and deployed automatically. The Representational State Transfer (REST) architectural style is commonly utilized by microservices to construct application programming interfaces (APIs) that are of the REST API name. Microservices communicate with one another through lightweight means. Since its inception in the year 2000, REST application programming interfaces (APIs) have been widely used by software companies in the process of developing their apps [10].

LITERATURE REVIEW

REST is the paradigm that is utilized the most frequently in the creation of microservices [11]. However, despite its widespread use, it does have some drawbacks when it comes to data access. These include the following: (i) query complexity (i.e., REST requires multiple HTTP requests to obtain multiple resources); (ii) over-fetching (i.e., a REST request returns more data than an application requires); and (iii) under-fetching (i.e., a particular endpoint does not give enough information, prompting additional requests to be made to obtain the required information), which is also referred to as the $n+1$ request problem [12]. To demonstrate the over-fetching and under-fetching issues that are associated with REST microservices, we took into consideration a scenario in which blogs were accessed by both a REST API and a GraphQL API [13].

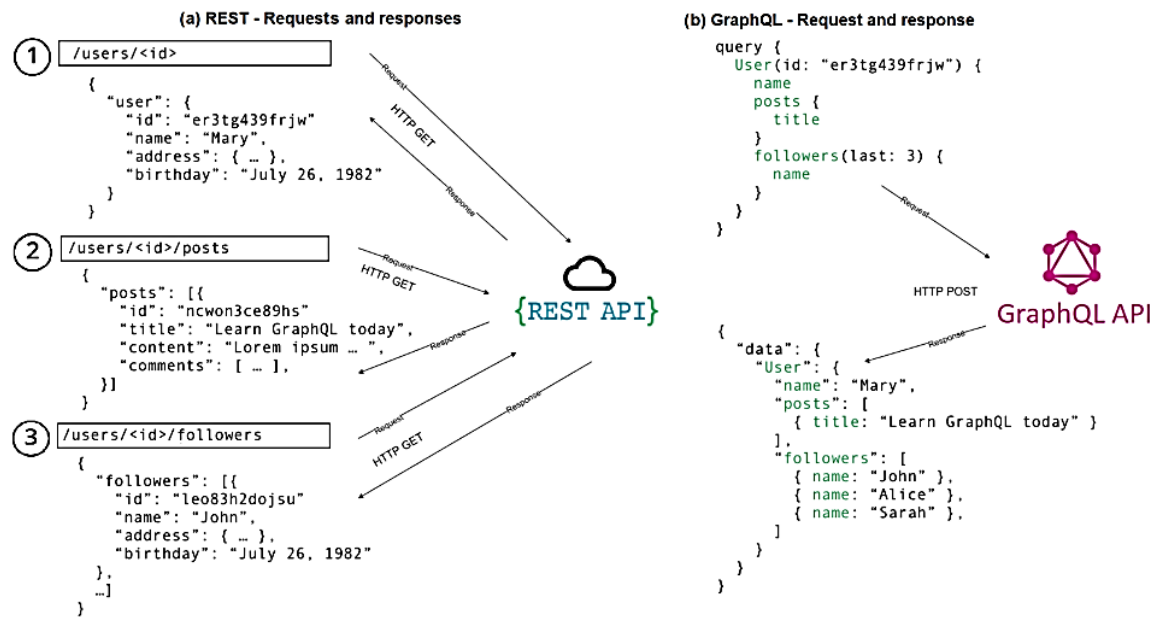


Fig. 1. Comparison of the REST API and the GraphQL API, including an example of over-fetching and under-fetching.

Figure 1 shows a client application that needs to find out who three users have followed a specific user in the last three months in addition to the user's post titles.

One argument's side shows the REST API request and response in Figure 1(a). Contacting various services, generally referred to as endpoints, is a typical way for REST to get data. To complete the provided scenario, it is crucial to query data at three endpoints. Data about users can be retrieved through the first endpoint, `/users/`. To access the user's postings, utilize the second endpoint, `/users//posts`. A list of all users' followers can be obtained via the third endpoint, which is `/users//followers`. In contrast to the preceding picture, picture 1(b) depicts the request and answer to the GraphQL API. According to this model, users can ask the server for specific data with a single query, and the server will only return that data [14]. In contrast to GraphQL, which only needs one endpoint access to get the necessary data, REST under-fetching happens when the request needs to access three endpoints to acquire the data it requires. In contrast, GraphQL only receives the data that is requested, whereas over-fetching occurs with REST when each endpoint access receives more data than is required.

Operational Semantics

GraphQL Language.

A complete syntax for document creation is available in the GraphQL language. Both the type system definitions and the actual queries that clients conduct on GraphQL services can be found on these pages. Hence, many definition instances, such as executables, type system extensions, or type system exemplifications, can be contained in a GraphQL document [15].

The primary components of each sort of definition are depicted in Figure 2, and the following will provide further explanation and examples of these components. Also, the type system of a GraphQL service can be described using the Integrated Development Language (IDL) that is a part of the GraphQL language. The tools make use of this IDL to provide advantages like service bootstrapping and client code generation.

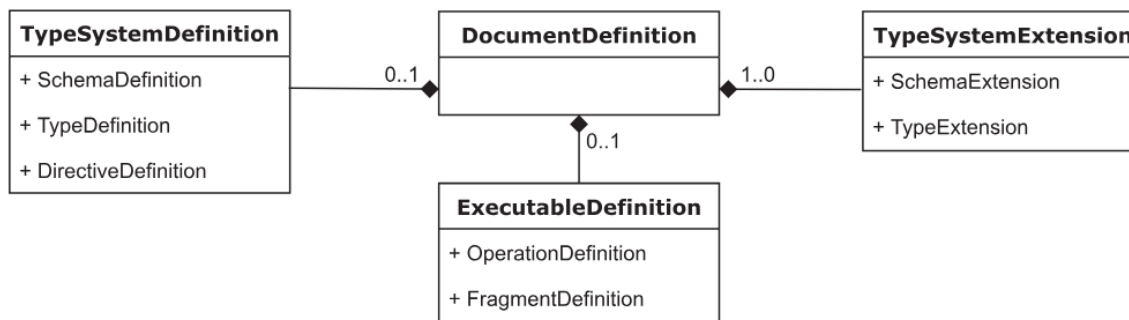


Fig. 2. GraphQL document definition.

Type System Definition

This is where the kinds of query variables' inputs and the data capabilities of a GraphQL server's Type System are defined.

Additional information is also provided. It is strictly forbidden for GraphQL execution services to carry out the execution of a GraphQL Document that contains a TypeSystemDefinition. Within the specification of the type system, we provide examples of both the conceptual structure and the interaction between the various components. The primary constituents have been divided into the following categories.

The schema defines the capabilities of a GraphQL service that are associated with the collective-type system. They are defined by a combination of types and directives; also, the following root operation types are linked to each sort of operation: query, mutation, and subscription.

Types are considered to be the most fundamental unit of the GraphQL schema. There are six recognized varieties of concrete and two wrapping types that can be used to characterize the different forms of concrete. The "named types" are as follows:

- Scalars: contains a primitive value that can be anything from text to an integer, float, Boolean, or even an ID.
- Enums: describe the set of possible values.
- Objects: A set of fields, each of which must correspond to a type in the system, must be defined. Fields are functions that take parameters to modify their behavior and return values. An implementation of a GraphQL server will usually only transform these arguments into a function. Values are returned by fields, which are functions. Consequently, Objects make up the intermediate layers of a response tree and Scalars and enums make up the leaves.

This allows for the development of arbitrary type hierarchies.

- Input objects: A set of input fields, including scalars, enums, and others, can be defined via this function.
- the object of the "named type":
- Interfaces: define a list of fields.
- Unions: To define a list of possible types. As a final point of discussion, we will refer to wrapping types as those that effectively enclose or wrap a "named type":

- List: Because of this, the list type is considered to be a wrapper for another type. A GraphQL schema may specify that a field is a representation of a list of other kinds.
- Non-null: is a wrapper for another type and indicates that the value that is produced will never be null.

Directives In a GraphQL document, provide a means by which alternative runtime execution and type validation behavior can be described. Additional information for types, fields, fragments, and operations can be described with the help of directives wherever they are required.

```

(a) schema {
  query: Query
  mutation: Mutation
}

(b) type Query {
  humans (id: Int!): [Human]
  droid (id: ID!): Droid
}

(c) type Mutation {
  createHuman (human: HumanInput): Human
}

(d) enum Episode {
  NEWHOPE
  EMPIRE
  JEDI
}

(e) type Character {
  name: String!
  appearsIn: [Episode]!
}

(f) interface Character {
  id: ID!
  name: String!
  friends: [Character]
  appearsIn: [Episode]!
}

(g) type Human implements Character {
  id: ID!
  name: String!
  friends: [Character]
  appearsIn: [Episode]!
  totalCredits: Int
}

(h) type Droid implements Character {
  id: ID!
  name: String!
  friends: [Character]
  appearsIn: [Episode]!
  primaryFunction: String
}

(i) input HumanInput {
  name: String!
  friends: [String]
  appearsIn: [String]!
  totalCredits: Int
}

(j) union SearchResult = Human | Droid

```

Fig. 3. GraphQL type system example.

Next, to support the idea behind the GraphQL paradigm specification, we give components some examples and demonstrations. Essential characters and plot points from the film are displayed here using a Star Wars API GraphQL service. An example of a type system definition with all the required components is shown in Figure 3 [16]: (a) Both the query and modification operations, as well as the GraphQL service schema, are described. (b) A list of type objects is returned by the query types (people, droid) when their arguments are of scalar type. (c) One kind of input is the type of the object that the mutation type creates returns. (d) The values indicated by the enum type (Episode) are specified. (e) There are several fields located inside the type object (Character).

ARCHITECTURAL STYLE

The client's ability to send and receive HTTP requests is facilitated by the APIs, with which GraphQL interacts. It employs a sophisticated query language for data extraction, retrieval, and modification, and organizes the data into a graph structure. Users can retrieve data from various resources with a single request using GraphQL. Compared to the alternative way of exchanging many requests, it makes API communication more easy and convenient.

When to use GraphQL

To facilitate the reuse, sharing, and effective monetization of bridge systems, GraphQL provides the digital connective tissue that is necessary. You may want to think about using GraphQL for your next project because it is useful in a variety of situations, including the following:

You should give GraphQL a go if you are a mobile developer and you need to optimize your app for higher bandwidth and faster performance. You want to be able to neatly encapsulate how data is fetched across various resources when you are working with complex and composite systems. This is because you do not want to indicate where the resources are located. GraphQL can be of assistance! If your platform only guides traffic to a single endpoint, GraphQL can accommodate a large number of resources that are being requested, with several microservices simultaneously processing each request. It is necessary to be quick while prototyping new applications. One way to speed up the development process is to incorporate an allowlisted GraphQL server.

Which types of applications can use GraphQL

These are some of the things that can be constructed using the data query language:

Applications that make use of different storage APIs to obtain data

software applications that need the retrieval of nested data in a single call

Applications designed for devices, such as smartwatches, mobile phones, and Internet of Things devices, where bandwidth utilization is a concern

What is REST?

The term REST stands for "Representational State Transfer," which is a form of software architecture that was developed to assist in the process of designing and developing the architecture for the World Wide Web. It gives end-users the ability to work with data over the internet with very little context from the outside world. By utilizing the standard HTTP methods, REST makes it possible for you to carry out fundamental operations on the data that is provided by the API itself. It does not make use of any resources or strategies, which makes it simple for REST APIs to handle many sorts of calls, exchange data in a variety of formats, and even change their structure by the hypermedia implementation.

The commands DELETE, PUSH, POST, and GET are among the most frequently used REST commands. Famous companies such as Twitter, Instagram, HubSpot, and Spotify all make use of REST application programming interfaces.

Fundamentals

End users can access and manipulate data from websites or web browsers thanks to its design paradigm, which is based on a uniform and specified set of stateless actions. The Application Programming Interfaces (APIs) in REST architecture showcase their functionality as resources, which in turn come with their Unique Resource Identifier, also known as URI. Sending a request to the server is how the client can gain access to the resource.

Pros

By utilizing REST, you can construct a monitoring system that is based on API answers.

Additionally, it is capable of handling a wide variety of call types and supports a wide range of data formats, such as plain text, JSON, and HTML.

You can make the client and server architectures independent of one another and provide excellent scalability for rapidly expanding applications.

APIs that use the REST protocol can handle a wide variety of queries and send data in a wide variety of formats. In other words, if the web application expands and adds more resources, REST will be able to handle the growing number of requests as well as the variety of queries.

A mature software architecture approach, REST has been around for decades and has been around for a long time. The fact that it is integrated into a number of the most widely used applications in the world, such as Twitter, Spotify, and Instagram, is evidence of its usefulness.

Cons

The structure of REST APIs does not adhere to any particular or standardized technique.

To retrieve all of the required data, REST must make many cycles of requests.

It has the potential to retrieve data without any context, which places a significant amount of strain on the applications because it is prone to under-fetching or over-fetching data.

Architectural style

Web services that adhere to the REST architectural style are referred to as RESTful web services. RESTful web services are defined as the set of principles that govern the creation of web services. Through the utilization of a predetermined set of guidelines, the arrangement makes it possible for a requesting system to gain access to and manipulate web resources.

When to use REST

With REST, you do not need to connect the application programming interface (API) to the client-side technology. Here are some examples of situations in which REST performs exceptionally well:

When working with web application programming interfaces (APIs), it makes use of HTTP. It is possible to use REST for almost any protocol with relative ease.

When there is an application that processes several requests for the same resource, REST makes it possible to cache the result of the request, which allows the server to be scaled up.

Which types of applications can use REST

A customer relationship management (CRM) application that must be developed for robust integrations can also benefit from REST. A software program that retrieves information about users and incorporates it into their system by utilizing REST APIs

An application for social sharing that could benefit from the use of REST APIs for the purpose of downloading and analyzing vast amounts of data sets

DATA OVER-FETCHING AND UNDER-FETCHING WITH GRAPHQL AND REST

For the sake of simplicity, over-fetching refers to the process by which a client downloads more information than is typically requested by the application. For example, if a screen needed to display a list of users with only their names, the application would hit the user's endpoint and retrieve a JSON object (user data) using REST. Such a screen would display the list of users.

With GraphQL, you can send a single query to the GraphQL server, which can include specific data requirements. This flexibility is available to you. The server provides a response in the form of a JSON object in which these requirements are satisfactorily satisfied. In a query, the client can provide the specific data that it requires.

Consequently, in the example, GraphQL will assist you in retrieving only the names of the users, rather than the complete data pertaining to them. When utilizing REST, it is necessary to send distinct requests to various endpoints to retrieve the necessary data sets.

Because the endpoints also respond with additional information, which is not required at all, you are engaged in excessive retrieval. Over-fetching, on the other hand, refers to the situation in which you do not have sufficient data with a call to an endpoint, which forces you to call a second endpoint. Both instances lead to problems with the performance of the system. Either you make more HTTP queries than is typically recommended or you consume more bandwidth than is requested.

Basic parameters in GraphQL and REST

Comparing GraphQL with REST: Which is superior? As far back as the API world is concerned, there has always been a heated discussion about the two. To tell you the truth, they are two distinct approaches that are used to address the same issue, which is to gain access to data available through web services. Let us take a look at X small differences that will convince you to choose one over the other because of their superiority:

Client-server architecture

Furthermore, this indicates that the client-server applications must operate separately. The client just needs to be aware of the Uniform Resource Locator (URI) of the information that is to be retrieved. It can't achieve communication with the server system in any other way. Regarding REST, the client and server designs develop independently of one another. The server is not aware of anything regarding the front-end user interface, and the client does not need to be aware of the business logic. When using GraphQL, things do not work out in that manner.

Stateless

Because all of the information that is required to correctly understand the request is contained inside the request itself, the server does not need to have any prior knowledge of the client state to correctly interpret the request when using GraphQL. In the case of REST, on the other hand, the client side is responsible for providing all of the information that is required to carry out a query. Additionally, servers are not allowed to store any information regarding a client's request or session.

Cacheability

The traceability of each response ought to be specified in the response itself. Without having to send the request to the server once more, the client will provide the data that it has stored in its cache in response to any request. For the REST framework to function properly, caching must be appropriately applied to resources. In client-server interactions, a caching system that is properly maintained eliminates a significant amount of bother. Both the app's performance and its availability are enhanced as a result of this. GraphQL clients can make use of HTTP caching to bypass the process of fetching resources and determining whether two resources are identical.

Layered system

In most cases, the architecture of an application is composed of numerous layers, with each layer specifically being aware of the layer that is immediately adjacent to it. At the same time, there may be a great deal of intermediary layers that exist between the client and the server. Through the use of load balancing, the arrangement enhances the availability of the system.

A client is unable to determine if it is directly linked to the final server in the system or merely the immediate one since REST makes it possible to employ a tiered system architecture to deploy the application programming interfaces (APIs) on Server A, stack and organize data on Server B, and authenticate user requests in Server C. If your application relies heavily on data that is connected, if you want to create an application programming interface (API) that can be used by several clients, or if you are just searching for a solution that can keep the amount of bandwidth that is being used under control, then GraphQL is the ideal data layer for your backend.

Uniform interface

Through the implementation of the following actions, you will be able to simplify the architecture by defining a uniform interface between the client and the server:

It is important that the resources be distinct from the representation that is communicated with the client and that they are recognized in the same way. Considering that the client possesses sufficient information to permit this, they must manage the resources through representations.

When communicating with the client, it is important to provide them with self-descriptive messages that include all of the information requested for processing. When compared to REST, GraphQL has an advantage because its interface is considered to be more straightforward. GraphQL queries produce predictable returns, which considerably improves the usefulness of the queries. Depending on the HTTP and URI techniques that are utilized, REST behaviors can differ.

CONCLUSIONS

Researchers and practitioners alike are taking notice of GraphQL, a fresh approach to API development. This work aims to provide a global perspective on the field by describing the so-called GraphQL paradigm from its formal specification. It does so by introducing a conceptual framework, which enables a thorough understanding of the paradigm and its components, and by demonstrating and describing these components. Then, we primarily focused on doing a literature review using a structured literature review (SMS) to provide a synopsis of the study, highlight current trends, and pinpoint any gaps in the GraphQL field. The seven research questions were addressed and the studies were categorized into general and specific GraphQL areas by the SMS. In our research, we found that GRAPHQL is becoming more popular as a preferred API management tool in scenarios where low API endpoint utilization and fast response times are critical performance indicators for managing large amounts of data. There is a learning curve to GRAPHQL, but as the tooling functions get better, the language will find many more uses in areas like business intelligence. We also recognize that REST is now a standard in the industry and that API management with REST endpoints has evolved.

REFERENCES

- [1] H. Arksey and L. O'Malley. 2005. Scoping studies: Towards a methodological framework. *Int. J. Soc. Res. Methodol.: Theory Pract.* 8, 1 (2005), 19–32. <https://doi.org/10.1080/1364557032000119616>
- [2] S. Baškarada, V. Nguyen, and A. Koronios. 2018. Architecting microservices: Practical opportunities and challenges. *J. Comput. Info. Syst.* 60, 5 (2018), 1–9. <https://doi.org/10.1080/08874417.2018.1520056>
- [3] P. Bourque and R. E. Fairley. 2014. SWEBOK v.3—Guide to the Software Engineering—Body of Knowledge. (3rd ed.). IEEE, 346 pages. <https://doi.org/10.1234/12345678>
- [4] R. Budde, K. Kautz, K. Kuhlenkamp, and H. Züllighoven. 1992. Prototyping An Approach to Evolutionary System Development. Springer-Verlag, Berlin. XII, 205 pages. <https://doi.org/10.1007/978-3-642-76820-0>
- [5] Clarivate. 2021. Journal Citation Reports: Reference Guide. Retrieved from https://clarivate.com/webofsciencigroup/content/uploads/sites/2/2021/06/JCR_2021_Reference_Guide.pdf.

- [6] Clarivate. 2021. Quartiles in JCR on the InCites Platform. Retrieved from <https://help.incites.clarivate.com/incitesLiveJCR/9053-TRS>.
- [7] N. Condori-Fernandez, M. Daneva, K. Sikkel, R. Wieringa, O. Dieste, and O. Pastor. 2009. A systematic mapping study on empirical evaluation of software requirements specifications techniques. In *Proceedings of the 3rd International Symposium on Empirical Software Engineering and Measurement (ESEM'09)* 1 (2009), 502–505. <https://doi.org/10.1109/ESEM.2009.5314232>
- [8] P. Di Francesco, P. Lago, and I. Malavolta. 2019. Architecting with microservices: A systematic mapping study. *J. Syst. Softw.* 150 (2019), 77–97. <https://doi.org/10.1016/j.jss.2019.01.001>
- [9] T. Dyba, T. Dingsoyr, and G. Hanssen. 2007. Applying systematic reviews to diverse study types: An experience report. In *Proceedings of the (ESEM'07)*. IEEE, 225–234. <https://doi.org/10.1109/ESEM.2007.59>
- [10] S. Easterbrook, J. Singer, M.-A. Storey, and D. Damian. 2008. Selecting empirical methods for software engineering research. *Guide Adv. Empir. Softw. Eng.* 1 (2008), 285–311. https://doi.org/10.1007/978-1-84800-044-5_11
- [11] R. Fresno-Aranda, P. Fernández, A. Durán, and A. Ruiz-Cortés. 2022. Semi-automated capacity analysis of limitationaware microservices architectures. In *Proceedings of the 19th International Conference on the Economics of Grids, Clouds, Systems, and Services (GECON'22)*. Springer, In press.
- [12] A. Galindo, D. Benavides, P. Trinidad, A. Gutiérrez-Fernández, and A. Ruiz-Cortés. 2019. Automated analysis of feature models: Quo vadis? *Computing* 101, 5 (2019), 387–433. <https://doi.org/10.1007/s00607-018-0646-1>
- [13] A. Gamez-Diaz, P. Fernandez, A. Ruiz-Cortés, P. J. Molina, N. Kolekar, P. Bhogill, M. Mohaan, and F. Méndez. 2019. The role of limitations and SLAs in the API industry. In *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, New York, NY, 1006–1014. <https://doi.org/10.1145/3338906.3340445>
- [14] J. M. García, O. Martin-Diaz, P. Fernandez, C. Müller, and A. Ruiz-Cortes. 2021. A flexible billing life cycle for cloud services using augmented customer agreements. *IEEE Access* 9 (2021), 44374–44389. <https://doi.org/10.1109/ACCESS.2021.3066443>

- [15] Google. 2021. FAQ about Trends. Retrieved from https://support.google.com/trends/answer/4365533?hl=en&ref_topic=6248052.
- [16] C. Guevara-Vega, B. Bernardez, A. Duran, A. Quina-Mera, M. Cruz, and A. Ruiz-Cortes. 2021. Empirical strategies in software engineering research: A literature survey. In Proceedings of the 2nd International Conference on Information Systems and Software Technologies (ICI2ST'21). IEEE, New York, NY, 120–127. <https://doi.org/10.1109/ICI2ST51859.2021.00025>.

Citation: Sudeesh Goriparthi. Enhancing data query efficiency with GraphQL APIs: A comparative study with RESTful web services. International Journal of Artificial Intelligence & Machine Learning (IJAIML), 3(1), 2024, pp. 132-143.

Article Link:

https://iaeme.com/MasterAdmin/Journal_uploads/IJAIML/VOLUME_3_ISSUE_1/IJAIML_03_01_013.pdf

Abstract Link:

https://iaeme.com/Home/article_id/IJAIML_03_01_013

Copyright: © 2024 Authors. This is an open-access article distributed under the terms of the Creative Commons Attribution License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited.

Creative Commons license: Creative Commons license: CC BY 4.0



✉ editor@iaeme.com